

CORE BUSINESS DO AntiGW

Objetivo: Verificar a autenticidade das alegações ambientais de produtos e criar histórico de verificações dos usuários.

ESTRUTURA DO BANCO DE DADOS (SQLite)

Tabela 1: usuarios

- id (INTEGER, PRIMARY KEY, AUTOINCREMENT)
- nome (TEXT, NOT NULL)
- email (TEXT, UNIQUE, NOT NULL)
- data_cadastro (TEXT/DATETIME)
- verificacoes_realizadas (INTEGER, DEFAULT 0)

Tabela 2: verificacoes

- id (INTEGER, PRIMARY KEY, AUTOINCREMENT)
- usuario_id (INTEGER, FOREIGN KEY -> usuarios.id)
- produto_nome (TEXT, NOT NULL)
- produto_barcode (TEXT)
- score_verde (INTEGER, 0-100)
- alertas_greenwashing (TEXT)
- data_verificacao (TEXT/DATETIME)

Tabela 3: produtos_cache (opcional, mas recomendada)

- barcode (TEXT, PRIMARY KEY)
 - nome (TEXT)
 - marca (TEXT)
 - score_calculado (INTEGER)
 - certificacoes (TEXT/JSON)
 - ingredientes_problematicos (TEXT)
 - ultima_atualizacao (TEXT/DATETIME)
-

ROTAS PRINCIPAIS DA API

CRUD de Usuários

1 POST /usuario

Propósito: Cadastrar novo usuário

Body (JSON):

```
{  
  "nome": "João Silva",  
  "email": "joao@email.com"  
}
```

Response: 201 Created + dados do usuário criado

2 GET /usuario/<id>

Propósito: Buscar dados de um usuário específico

Response: Dados do usuário + total de verificações

3 GET /usuarios

Propósito: Listar todos os usuários (para admin/dashboard)

Response: Lista de todos os usuários

4 DELETE /usuario/<id>

Propósito: Deletar usuário e suas verificações

Response: 200 OK confirmando deleção

CORE BUSINESS - Verificações de Produtos

5 POST /verificar_produto ★ ROTA PRINCIPAL

Propósito: Verificar se produto tem greenwashing

Body (JSON):

```
{
  "usuario_id": 1,
  "produto_nome": "Sabão Eco Green",
  "barcode": "7891234567890"
}
```

Lógica:

- Consulta Open Food Facts API
- Calcula score verde (0-100)
- Identifica alertas de greenwashing
- Salva verificação no banco
- Atualiza contador do usuário

Response:

```
{
  "id": 15,
  "produto_nome": "Sabão Eco Green",
  "score_verde": 45,
  "status": "ATENÇÃO - Possível Greenwashing",
  "alertas": [
    "Sem certificações ambientais válidas",
    "Contém ingredientes não biodegradáveis"
  ],
  "data_verificacao": "2025-11-14 10:30:00"
}
```

6 GET /historico/<usuario_id>

Propósito: Listar todas as verificações de um usuário

Response: Array de verificações ordenadas por data

7 GET /verificacao/<id>

Propósito: Buscar detalhes de uma verificação específica

Response: Dados completos da verificação

8 DELETE /verificacao/<id>

Propósito: Deletar uma verificação do histórico

Response: 200 OK confirmando deleção

EXTRAS (Diferenciais)

9 GET /estatisticas/<usuario_id>

Propósito: Dashboard do usuário

Response:

```
{
  "total_verificacoes": 23,
  "produtos_confiveis": 8,
  "greenwashing_detectado": 15,
  "score_medio_produtos": 52,
  "top_alertas": [
    "Sem certificações: 10 produtos",
    "Ingredientes não sustentáveis: 7 produtos"
  ]
}
```

FLUXO DO USUÁRIO (SPA)

1. **Página inicial:** Formulário de busca de produto
 2. **Ação:** Usuário digita nome do produto → POST /verificar_produto
 3. **Resultado:** Card mostrando score verde + alertas
 4. **Navegação:** Lista com "Meu Histórico" → GET /historico/<usuario_id>
 5. **Interação:** Pode deletar verificações antigas → DELETE /verificacao/<id>
 6. **Dashboard:** Visualizar estatísticas gerais → GET /estatisticas/<usuario_id>
-

ALGORITMO DE SCORE VERDE (simplificado para MVP)

```
def calcular_score_verde(produto_data):
    score = 100
```

```

alertas = []

# Penalizações
if not produto_data.get('certificacoes'):
    score -= 30
    alertas.append("Sem certificações ambientais válidas")

if 'palm oil' in produto_data.get('ingredients', '').lower():
    score -= 20
    alertas.append("Contém óleo de palma (desmatamento)")

if produto_data.get('packaging') == 'plastic':
    score -= 15
    alertas.append("Embalagem não reciclável")

# Bonificações
if 'organic' in produto_data.get('labels', []):
    score += 10

return max(0, min(100, score)), alertas

```

RESUMO DAS ROTAS OBRIGATÓRIAS (Requisito: mínimo 4)

- ✓ POST /usuario - Cadastrar Usuário
- ✓ GET /usuario/<id> - Buscar um Usuário
- ✓ GET /usuarios - Buscar todos
- ✓ DELETE /usuario/<id> - Deletar
- ✓ POST /verificar_produto - **CORE BUSINESS**
- ✓ GET /historico/<usuario_id> - Listar verificações
- ✓ DELETE /verificacao/<id> - Deletar verificação

Total: 7 rotas (3 além do mínimo = pontos extras de criatividade) 🎯

Quer que eu detalhe mais alguma parte específica? Posso ajudar com:

- Estrutura de pastas do projeto
- Código inicial do Flask
- Lógica de integração com Open Food Facts

- Sugestões de UI/UX para o frontend